

# State-Aware Adaptation and Control of Foundation Models

Yong Zhong | Research Interests | August 2026

## Where the question came from

My first research project, RADAR at SUSTech (IROS 2026, co-first author), is a closed-loop data generation engine for manipulation. A vision-language model proposes tasks, a policy executes them, the model judges success, and a finite state machine handles reset, data routing, and recovery from execution failures. I integrated the policy into that loop, built the RLBench scenes, and ran most of the simulation evaluation. Running the system repeatedly made me wonder whether knowing that a policy had failed was enough. The pipeline detects failure and recovers from it, but it does not explicitly use why the policy failed to decide what data should be collected next, and I began to ask whether adaptation should depend on how something failed, not only on whether it failed.

## Runtime adaptation as the tool

Since February 2026 I have been a visiting research student at NUS, advised by Kai Wang and Prof. Yang You. I came to work on parameter generation with hypernetworks because generating or selecting parameters at inference time is the most direct way to let a model change while it runs. The question I keep returning to is whether a model can use its own state to decide when, where, and how much to change itself or its behavior. Two projects turned this into concrete problems.

## Representation level, my first-author steering project

On the steering benchmarks I was using, representation editing often lagged prompting, and a strong recent method on those benchmarks intervened at a single layer. That made me ask whether the problem was not only what representation to add, but when and where to add it. We first tried editing the KV cache to exploit cache reuse. It underperformed, so the method moved to hidden-state interventions generated per token and per layer from the live residual stream and a target behavior. One fixed intervention turned out to be too coarse, because its effect depends on the token, the layer, and the model's evolving state. The next step is a closed loop, a controller that observes the consequence of its previous intervention and decides whether, where, and how strongly to intervene again, with locality and stability measured alongside behavioral control.

## Parameter level, a diffusion language model project I contribute to

The second project studies timestep-conditioned parameter-efficient adaptation for masked diffusion language models, where early denoising steps face high uncertainty and later steps need precise refinement. In my early experiments I explored a mixer and dynamic gating between adapter components. Those mechanisms did not survive into the final design. The negative result was useful because it narrowed the question from how much machinery we could add to what actually needed to vary across inference states. The question I want to pursue is whether adaptation to internal inference state and robustness to external model changes can be separated. CAR-LoRA (Prof. Wei's group, ICLR 2026) trains one adapter to remain effective after quantization, pruning, or base-model updates. A first experiment would cross compressed or updated base models with denoising-state bins and compare a fixed adapter, a robust adapter, state-dependent routing alone, and a factored design with a shared component trained for robustness and a small state-dependent component selecting the current update. The factored arm is the actual test. Its answer would also tell a parameter generator what it must produce for a new task or deployment domain and what can stay fixed.

## Where this goes, the level of external behavior

The same question applies to what an agent does next. If internal state predicted trouble early, an agent could verify an observation, revise a plan, switch tools, roll back, or stop before an error becomes visible in the environment, and sometimes a correction inside the model would be cheaper than an external action. A data pipeline like RADAR could likewise collect differently after different failures. Prof. Wei's recent work frames this from the reward and environment side. SELAUR injects step-level and trajectory-level uncertainty into agent rewards, and the sim-to-real perspective on foundation-model agents sorts deployment failures into observation, action, transition, and reward mismatches. A first test would ask whether uncertainty or representation drift predicts which type of mismatch is about to occur, and whether acting on that prediction reduces unnecessary actions and long-horizon errors at a fixed budget.

## Scope

Across these problems I would compare adaptive methods with fixed adapters, static steering, prompting, external routing, and full fine-tuning under controlled changes in model, inference state, and environment, and measure locality, stability, and intervention cost alongside task performance and generalization. In the long run I would like to understand whether these decisions can be guided by a common state-aware principle, but I do not expect to solve all three levels at once. I would rather begin with one problem where I can establish a clear mechanism and reliable evidence, then decide from that whether the same principle transfers.